Hashing and imports

```
from dataclasses import dataclass, field
from typing import Any

DIFFICULTY = 2


def sha256_hex(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def canonical(obj: object) -> str:
    return json.dumps(obj, sort_keys=True, separators=(',', ':'))


print('DIFFICULTY =', DIFFICULTY)
print('sha256_hex(b"mZAR") =', sha256_hex(b'mZAR')[:16], '...')
```

```
DIFFICULTY = 2
sha256_hex(b"mZAR") = d3b93a230d27eaa0 ...
```

Toy wallets

```
    def __post_init__(self) -> None:
        self.sk = sha256_hex(f'toy-sk-{self.label}'.encode())
        self.pk = sha256_hex(f'toy-pk-{self.sk}'.encode())
        self.address = self.pk[:40]


def create_wallets() -> tuple[Wallet, Wallet, Wallet]:
    """Return sender, recipient, and optional reserve agent."""
    return Wallet('Sender'), Wallet('Recipient'), Wallet('ReserveAgent')


sender, recipient, reserve = create_wallets()
print('Sender   ', sender.address)
print('Recipient', recipient.address)
print('Reserve  ', reserve.address)
```

```
Sender    cc6154ce75ed89ab8dea0b12e45866c988f513ef
Recipient 56f62e486f82191a14f9f6ecaaa5329d0addfd02
Reserve   28312b71e08138db7b7cb1ce7f14255cf9ea71a7
```

Reserve ledger

```
            raise ValueError('insufficient balance')
        self.balances[address] -= amount
        self.circulating -= amount
        self.vault_fiat -= amount
        receipt = {
            'receipt_id': sha256_hex(f'{address}:{amount}:{time.time()}'.encode())[:16],
            'address': address,
            'paid_local': amount,
        }
        self.receipts.append(receipt)
        return receipt


# Smoke: starter test_onramp_invariant
_led = ReserveLedger()
_led.onramp('addrA', 1000)
assert _led.invariant_ok()
assert _led.balances['addrA'] == 1000
assert _led.vault_fiat == 1000
print('ReserveLedger smoke OK — invariant', _led.invariant_ok())
```

```
ReserveLedger smoke OK — invariant True
```

Sign/Verify

```
        'sender': frm.address,
        'recipient': to_addr,
        'amount': amount,
        'timestamp': int(time.time()),
        'asset': asset,
    }
    tx['signature'] = sign(tx, frm.sk)
    return tx


sample = build_transfer(sender, recipient.address, 10)
assert verify_tx(sample, sender)
tampered = dict(sample)
tampered['amount'] = 9999
assert not verify_tx(tampered, sender)
print('sign/verify OK; tamper rejected')
```

```
sign/verify OK; tamper rejected
```

Chain helpers

```
        if block_hash(block) != block['hash']:
            return False
        if i and block['previous_hash'] != chain[i - 1]['hash']:
            return False
    return True


def make_genesis() -> dict:
    return mine_block([{'note': 'genesis'}], '0' * 64, 0)


genesis = make_genesis()
assert verify_chain([genesis])
print('Genesis mined index=', genesis['index'], 'hash=', genesis['hash'][:16], '...')
print('verify_chain OK')
```

```
Genesis mined index= 0 hash= 001e425a07c4edd2 ...
verify_chain OK
```

Simulated on-ramp

```
#Simulated on-ramp

ledger = ReserveLedger()
chain: list[dict] = [make_genesis()]
mempool: list[dict] = []

LOCAL_DEPOSIT = 1000
TRANSFER_AMOUNT = 750

ledger.onramp(sender.address, LOCAL_DEPOSIT)
assert ledger.invariant_ok()

print('ON-RAMP')
print('  vault_fiat   =', ledger.vault_fiat)
print('  circulating  =', ledger.circulating)
print('  sender bal   =', ledger.balances[sender.address])
print('  invariant_ok =', ledger.invariant_ok())
```

```
ON-RAMP
  vault_fiat   = 1000
  circulating  = 1000
  sender bal   = 1000
  invariant_ok = True
```

On chain transfer

```
tx750 = build_transfer(sender, recipient.address, TRANSFER_AMOUNT)
assert verify_tx(tx750, sender)
accepted = accept_tx(tx750, ledger, mempool)
print('ACCEPT into mempool:', accepted, '| mempool size:', len(mempool))

block = mine_pending(ledger, chain, mempool)
print('MINED block', block['index'], 'hash', block['hash'][:16], '... nonce', block['nonce'])
print('BALANCES after mine — sender', ledger.balances[sender.address],
      '| recipient', ledger.balances[recipient.address],
      '| vault', ledger.vault_fiat)
assert ledger.vault_fiat == LOCAL_DEPOSIT  # transfer does not touch vault
assert ledger.invariant_ok()
assert verify_chain(chain)
```

```
ACCEPT into mempool: True | mempool size: 1
MINED block 1 hash 000fb269b35d2125 ... nonce 27
BALANCES after mine — sender 250 | recipient 750 | vault 1000
```

Confirmation depth for payouts

```
assert h_pay is not None
k_now = confirmation_depth(chain, h_pay)
print(f'Payment included at height h={h_pay}; tip H={chain[-1]["index"]}; depth k={k_now}')
print('ready_for_offramp (k_min=1):', ready_for_offramp(chain, h_pay, 1))

# Illustrate waiting for deeper confirmations (optional policy)
MIN_K_DEMO = 2
while not ready_for_offramp(chain, h_pay, MIN_K_DEMO):
    mine_empty_for_depth(chain, 1)
    print('  padded -> tip', chain[-1]['index'],
          'k=', confirmation_depth(chain, h_pay))

print(f'After padding: k={confirmation_depth(chain, h_pay)} >= {MIN_K_DEMO} -> payout allowed')
assert verify_chain(chain)
```

```
Payment included at height h=1; tip H=1; depth k=1
ready_for_offramp (k_min=1): True
  padded -> tip 2 k= 2
After padding: k=2 >= 2 -> payout allowed
```

Simulated off ramp

```
# Simulated off ramp

assert ready_for_offramp(chain, h_pay, min_confirmations=1)

receipt = ledger.offramp(recipient.address, TRANSFER_AMOUNT)
assert ledger.invariant_ok()
assert ledger.vault_fiat == LOCAL_DEPOSIT - TRANSFER_AMOUNT  # 250
assert ledger.balances.get(recipient.address, 0) == 0
assert ledger.balances[sender.address] == LOCAL_DEPOSIT - TRANSFER_AMOUNT

print('OFF-RAMP receipt:', receipt)
print('vault_fiat =', ledger.vault_fiat, '| circulating =', ledger.circulating)
print('invariant_ok =', ledger.invariant_ok())
```

```
OFF-RAMP receipt: {'receipt_id': '7345c45ffa5fb693', 'address': '56f62e486f82191a14f9f6ecaaa5329d0addfd02', 'paid_local': 750}
vault_fiat = 250 | circulating = 250
invariant_ok = True
```

Balance accounting demo path

```
demo_led.offramp(r2.address, 750)
rows.append(snapshot(demo_led, 'off-ramp 750', s2.address, r2.address))

print(f"{'step':<16} {'vault':>6} {'circ':>6} {'sender':>7} {'recip':>7} {'inv':>5}")
print('-' * 52)
for row in rows:
    print(f"{row['step']:<16} {row['vault']:6d} {row['circulating']:6d} "
          f"{row['sender']:7d} {row['recipient']:7d} {str(row['invariant']):>5}")

assert all(r['invariant'] for r in rows)
assert rows[-1]['vault'] == 250 and rows[-1]['sender'] == 250 and rows[-1]['recipient'] == 0
print('\nAccounting path OK')
```

```
step              vault   circ  sender   recip   inv
----------------------------------------------------
start                 0      0       0       0  True
on-ramp 1000       1000   1000    1000       0  True
transfer 750       1000   1000     250     750  True
off-ramp 750        250    250     250       0  True

Accounting path OK
```

Reject invalid transfer

```
#Reject invalid transfer

# Fresh mini-state so rejection is independent of prior balances
rej_led = ReserveLedger()
rej_led.onramp(sender.address, 1000)
rej_pool: list[dict] = []

bad = build_transfer(sender, recipient.address, 10)
bad['amount'] = 9999  # mutate after sign — signature no longer matches
tamper_rejected = not accept_tx(bad, rej_led, rej_pool)

overspend = build_transfer(sender, recipient.address, 5000)  # valid sig, insufficient funds
overspend_rejected = not accept_tx(overspend, rej_led, rej_pool)

print('tampered accept?', not tamper_rejected, '-> rejected?', tamper_rejected)
print('overspend accept?', not overspend_rejected, '-> rejected?', overspend_rejected)
print('mempool still empty?', len(rej_pool) == 0)
assert tamper_rejected and overspend_rejected and len(rej_pool) == 0
print('Invalid transfer rejection OK')
```

```
tampered accept? False -> rejected? True
overspend accept? False -> rejected? True
mempool still empty? True
Invalid transfer rejection OK
```

Explorer style printout

```
    for b in ch:
        print(f"Block {b['index']} {b['hash'][:12]}... txs={len(b['transactions'])}")
        for t in b['transactions']:
            if isinstance(t, dict) and 'amount' in t:
                print(f"    {t.get('sender', '?')[:8]}... -> {t.get('recipient', '?')[:8]}... "
                      f"amount={t['amount']} {t.get('asset', '')}")
            else:
                print('    ', t)


print_explorer(chain)
print('chain_ok =', verify_chain(chain))
```

```
Block 0 00e3f0872b6c... txs=1
    {'note': 'genesis'}
Block 1 000fb269b35d... txs=1
    cc6154ce... -> 56f62e48... amount=750 mZAR
Block 2 00505bed052e... txs=1
    {'note': 'confirmation-padding-2'}
chain ok = True
```

End to End remittance corridor

```
out = run_remittance_demo()
summary = {k: out[k] for k in out if k != 'explorer'}
print(json.dumps(summary, indent=2))
print('blocks:', len(out['explorer']))

assert out['invariant'] is True
assert out['chain_ok'] is True
assert out['invalid_rejected'] is True
assert out['vault_fiat'] == 250
assert out['circulating'] == 250
print('\nEnd-to-end remittance corridor OK')
```


```
{
  "balances": {
    "cc6154ce75ed89ab8dea0b12e45866c988f513ef": 250,
    "56f62e486f82191a14f9f6ecaaa5329d0addfd02": 0
  },
  "vault_fiat": 250,
  "circulating": 250,
  "invariant": true,
  "chain_ok": true,
  "receipt": {
    "receipt_id": "29a8c8733d8d85d1",
    "address": "56f62e486f82191a14f9f6ecaaa5329d0addfd02",
    "paid_local": 750
```

```
  },
  "invalid_rejected": true,
  "confirmation_depth": 1,
  "payment_height": 1
}
blocks: 2

End-to-end remittance corridor OK
```

Mvp checklist

```
print('-' * 72)
for label, done, note in checks:
    mark = '[x]' if done else '[ ]'
    print(f'  {mark} {label:<42} {note}')

core_ok = all(done for label, done, _ in checks if not label.startswith('(f)'))
assert core_ok
print('\nCore MVP items (a-e, g) satisfied in-process; (f) via optional Flask cell.')
```

```
A9 MVP checklist (in-process path)
------------------------------------------------------------------------
  [x] (a) Wallets for parties                  create_wallets() / Wallet dataclass
  [x] (b) On-ramp at 1:1                       onramp then offramp left vault=250
  [x] (c) Signed transfer + mine               750 mZAR in mined block
  [x] (d) Off-ramp + receipt                   29a8c8733d8d85d1
  [x] (e) Verification (verify_chain + lookup)   payment_height=1
  [ ] (f) Payment API                          optional Flask stretch below / wrap A8 routes
  [x] (g) Explorer view                        2 blocks summarised

Core MVP items (a-e, g) satisfied in-process; (f) via optional Flask cell.
```